

```
import numpy as np
import pandas as pd

LB = 576           # 2-day swing window for structural lows
OI_LB = 1152       # 4-day OI expansion window
CVD_SLOPE_LB = 288

# 1. Price structure: confirmed lower lows
df['roll_low'] = df['Low'].rolling(LB).min()
df['prev_roll_low'] = df['roll_low'].shift(LB)
price_ll = df['roll_low'] < df['prev_roll_low'] # price making lower lows

# 2. OI structure: higher highs over the SAME window
df['oi_max'] = df['Agg. OI'].rolling(OI_LB).max()
df['prev_oi_max'] = df['oi_max'].shift(OI_LB)
oi_HH = df['oi_max'] > df['prev_oi_max'] * 1.05 # require >=5% OI expansion

# 3. Normalize the divergence: OI rising while price falling, measured as
# correlation of detrended price returns vs OI returns over the window
```

```
df['ret'] = df['Close'].pct_change()
df['oi_ret'] = df['Agg_OI'].pct_change()
df['div_corr'] = (df['ret'].rolling(OI_LB)
                  .corr(df['oi_ret']))

oi_div = df['div_corr'] < -0.25 # negative coupling = divergence

# 4. CVD absorption: CVD slope must NOT confirm the price decline
df['cvd_slope'] = df['CVD'].diff(CVD_SLOPE_LB)
cvd_absorb = df['cvd_slope'] >= 0 # CVD flat/up while price down

regime_active = price_ll & oi_hh & oi_div & cvd_absorb
df['regime_SAOID'] = regime_active.astype(bool)
```

Copy

The economic claim each line encodes: lower lows (distress) + $\geq 5\%$ OI build (conviction inflow) + negative price/OI correlation (the build is *against* price, not chasing it) + non-negative CVD slope (taker buyers absorbing the sellers).

The Entry Trigger



You do not enter on the divergence itself — that's catching a falling knife. You enter on **the resolution**: the first close that reclaims the divergence range *with a CVD impulse confirming the latent positioning is now expressing*.

```
TRIGGER_LB = 288
df['range_high'] = df['High'].rolling(TRIGGER_LB).max().shift(1)
df['cvd_impulse'] = df['CVD'].diff() > df['CVD'].diff().rolling(288).std() * 2.0

entry_signal = (df['regime_SAOID'].rolling(TRIGGER_LB).max().astype(bool) # regime seen
recently
                & (df['Close'] > df['range_high'])) # breakout
reclaim       & df['cvd_impulse']) # CVD confirms
```

Copy

Risk Management

Initial structural stop: placed below the accumulation low — this is the entire thesis-invalidation point. If price loses the low the OI was built against, the accumulation was actually trapped longs, and the cascade is now *against* you.

```
initial_stop = roll_low_at_entry * (1 - 0.002) # 20bps buffer below structural low
```

Copy



Trailing mechanism — Chandelier Exit (ATR-anchored to highest high): this is the correct trailer here because the payoff is a violent expansion move and you want to give it room proportional to realized volatility.

```
ATR_n = 22 # ~110 min ATR
mult = 3.5 # wide multiple to ride asymmetric payout
chandelier_stop[i] = max(High over since-entry) - mult * ATR[i]
stop[i] = max(stop[i-1], chandelier_stop[i]) # ratchet only upward
```

Copy

The $3.5 \times$ ATR multiple is deliberate: a tighter trailer would get noised out before the move pays the 8 bps fee plus delivers your target asymmetry.

Architecture 2 — Volatility Contraction & Funding Apathy Breakout (VCFAB)

The Core Quantitative Logic

The inefficiency **funding rate is a real-time sentiment tax**. When Agg. Funding Rate collapses toward zero *and* True Range compresses simultaneously, you have a market where neither longs nor shorts are paying to hold a directional bet — complete positioning apathy. This is the perp-market analogue of a volatility squeeze, but with a sentiment dimension that spot markets lack. Energy is being stored. Apathetic, balanced order books are structurally fragile to a one-sided institutional taker sweep because there is no opposing leveraged crowd to absorb it. The trade is to position on the *first* large Candle Delta impulse that breaks the coil, because that impulse is the institution that decided the apathy was mispriced.



The Trend Filter (Pandas)

```
ATR_n = 14
df['TR'] = np.maximum.reduce([
    (df['High'] - df['Low']).values,
    (df['High'] - df['Close']).shift().abs().values,
    (df['Low'] - df['Close']).shift().abs().values
])
df['ATR'] = df['TR'].rolling(ATR_n).mean()

# 1. Volatility contraction: ATR in its own low percentile over a quarter (~3 months)
VOL_LB = 25920 # 90 days of 5-min bars
df['atr_pct'] = df['ATR'].rolling(VOL_LB).rank(pct=True)
vol_contract = df['atr_pct'] < 0.15 # bottom 15% of realized vol

# 2. Funding apathy: [funding] near zero relative to its own distribution
df['fund_abs'] = df['Agg. Funding Rate'].abs()
df['fund_pct'] = df['fund_abs'].rolling(VOL_LB).rank(pct=True)
fund_apathy = df['fund_pct'] < 0.28 # bottom 28% [funding]

# 3. The coil must persist ~ require contraction sustained, not a one-bar dip
COIL_PERSIST = 288
regime_active = ((vol_contract & fund_apathy)
                  .rolling(COIL_PERSIST).mean() > 0.70) # apathy held 70% of last day
df['regime_VCFAB'] = regime_active.astype(bool)
```

Copy



The Entry Trigger

The breakout direction is *chosen by the Candle Delta impulse itself* — this is direction-agnostic until the institution reveals its hand.

```
df['delta_z'] = ((df['Candle Delta'] - df['Candle Delta'].rolling(288).mean())
                / df['Candle Delta'].rolling(288).std())

# Squeeze boundaries captured at the last bar the regime was active
df['coil_high'] = df['High'].rolling(COIL_PERSIST).max().shift(1)
df['coil_low'] = df['Low'].rolling(COIL_PERSIST).min().shift(1)

long_entry = (df['regime_VCFAB'].shift(1)
              & (df['Close'] > df['coil_high'])
              & (df['delta_z'] > 3.0)) # 3-sigma BUY delta impulse
short_entry = (df['regime_VCFAB'].shift(1)
              & (df['Close'] < df['coil_low'])
              & (df['delta_z'] < -3.0)) # 3-sigma SELL delta impulse
```

Copy

Risk Management

Initial structural stop: the opposite boundary of the **CVD**. The squeeze thesis is invalidated if price re-enters and crosses the full range — that means the breakout was a false flush.

```
long: initial_stop = coil_low * (1 - 0.001)
short: initial_stop = coil_high * (1 + 0.001)
```

Trailing mechanism — Parabolic SAR (accelerating): the SAR is the right tool for a *post-squeeze expansion* because volatility is now rising from a known-low base, and the accelerating step lets you tighten as the move matures and matches the fact that post-coil moves often trend hard then snap.

```
AF_start = 0.02, AF_step = 0.02, AF_max = 0.20
SAR[i] = SAR[i-1] + AF * (EP - SAR[i-1])
where EP = extreme price since entry, AF increments by AF_step each new EP up to AF_max
```

Because vol was compressed at entry, the initial SAR sits tight relative to the eventual expansion, giving favorable R-multiple geometry to clear fee drag.

Architecture 3 — The Structural Pullback (Mean Reversion to Trend) (SPMRT)

The Core Quantitative Logic

The inefficiency, in an established macro uptrend, the highest-edge entry is not the breakout (already extended, poor R:R after fees) but the **first pullback where passive bid absorption defeats taker selling**. On perps, you can see this absorption directly via CVD: if price prints lower while CVD rises or holds (sellers hitting bids that don't move price down proportionally), liquidity providers are absorbing the dip. That divergence between falling price and rising/flat CVD is the footprint of institutional limit-bid accumulation inside the trend — the cleanest "buy the dip" signal that exists in this dataset.

The Trend Filter (Pandas)

```
# Macro trend via 200 EMA - but on 5-min, use a multi-day EMA span
EMA_SPAN = 1152 # ~4-day EMA acting as the macro "200 EMA" equivalent
df['EMA_macro'] = df['Close'].ewm(span=EMA_SPAN, adjust=False).mean()

# Trend must be both above EMA AND positively sloped
df['ema_slope'] = df['EMA_macro'].diff(288) # 1-day slope
uptrend = (df['Close'] > df['EMA_macro']) & (df['ema_slope'] > 0)

# Require trend maturity: price has been above EMA for a sustained period
df['regime_SPMRT'] = (uptrend.rolling(2880).mean() > 0.80) # above EMA 80% of last 10 days
```

The Entry Trigger

Identify the pullback (price retraces toward EMA) and require CVD absorption on the dip.

```
# Pullback: price has dipped toward the EMA but not broken it
df['dist_to_ema'] = (df['Close'] - df['EMA_macro']) / df['EMA_macro']
pullback_zone = (df['dist_to_ema'] > 0) & (df['dist_to_ema'] < 0.01) # within 1% above EMA

# Absorption divergence over the pullback leg
PULL_LB = 36 # 3-hour pullback measurement
df['price_chg'] = df['Close'].diff(PULL_LB)
df['cvd_chg'] = df['CVD'].diff(PULL_LB)
absorption = (df['price_chg'] < 0) & (df['cvd_chg'] > 0) # price down, CVD up = absorbed

# Optional RSI confirmation: pullback into oversold-within-uptrend
rsi_dip = df['RSI'] < 45

# Entry - first bar reclaiming after absorption inside the regime
df['reclaim'] = df['Close'] > df['Close'].shift(1)
entry_signal = (df['regime_SPMRT']
                & pullback_zone
                & absorption
                & rsi_dip
                & df['reclaim'])
```

Risk Management

Initial structural stop: below the macro EMA itself — the entire premise is "trend intact." If price closes meaningfully below the 4-day EMA, the trend regime is broken and the pullback was a reversal.

```
initial_stop = EMA_macro_at_entry * (1 - 0.005) # 50bps below the macro EMA
```

Trailing mechanism — Donchian Channel (lower band): Donchian is the correct trailer for a *resumption-of-trend* entry because it locks gains to recent structure (higher lows) and only stops you when the trend makes a genuine lower low — which is exactly the trend-invalidation event you care about.

```
DONCHIAN_n = 96 # 8-hour channel
donchian_low[i] = min(Low over last 96 bars)
stop[i] = max(stop[i-1], donchian_low[i]) # ratchet upward only
```

This lets winners run to the multi-R asymmetric payouts you need, exiting only on a true structural break of higher lows.

The Shared Numba Execution Loop

All three plug into one compiled engine. Precompute every indicator in Pandas, export arrays, then:

```
import numba as nb

@nb.njit(cache=True)
def run_backtest(open_, high, low, close,
                  entry_long, entry_short,
                  init_stop_arr, atr, fee=0.0008):
    n = close.shape[0]
    equity = np.ones(n)
    position = 0 # 0 flat, 1 long, -1 short
    entry_px = 0.0
    stop = 0.0
    extreme = 0.0
    eq = 1.0
    mult = 3.5
```

```

for i in range(1, n):
    if position == 0:
        if entry_long[i]:
            position = 1
            entry_px = close[i]
            stop = init_stop_arr[i]
            extreme = high[i]
            eq *= (1.0 - fee) # entry fee
        elif entry_short[i]:
            position = -1
            entry_px = close[i]
            stop = init_stop_arr[i]
            extreme = low[i]
            eq *= (1.0 - fee)

    elif position == 1:
        if high[i] > extreme:
            extreme = high[i]
            trail = extreme - mult * atr[i] # Chandelier example
            if trail > stop:
                stop = trail
            if low[i] <= stop: # stop hit
                eq *= (stop / entry_px)
                eq *= (1.0 - fee) # exit fee
                position = 0

    elif position == -1:
        if low[i] < extreme:
            extreme = low[i]
            trail = extreme + mult * atr[i]
            if trail < stop:
                stop = trail
            if high[i] >= stop:
                eq *= (entry_px / stop)
                eq *= (1.0 - fee)
                position = 0

    equity[i] = eq
return equity

```

Swap the `trail` line for the SAR recursion or Donchian band to match each architecture (those are stateful, so you carry `sar`, `af`, or maintain a rolling-min buffer inside the loop). Keep all inputs as `np.float64/np.bool_` contiguous arrays — that's what makes `cache=True` compilation stable.

A few things I'd flag as a colleague rather than leave you to discover in production. First, **look-ahead leakage is the silent killer here**: `.rolling().rank(pct=True)` and any `.shift()` boundaries need an audit to confirm no future bar leaks into a signal — backtest every entry as if you only knew data up to bar `i-1`. Second, **Agg. OI and CVD are exchange-aggregated and frequently revised or gapped**, a divergence built on a data artifact will look like alpha and trade like noise, so validate against raw venue data on a sample. Third, the parameter counts (windows, multiples, sigma thresholds) are large enough that **overfitting risk is severe** — these need walk-forward optimization with strict out-of-sample holdout, not a single in-sample sweep. Finally, the 3-sigma and percentile thresholds I chose are reasonable priors, not tuned values, expect to re-derive them per asset, since BTC, ETH, and alt perps have very different funding and OI regimes.

Want me to write out the exact stateful SAR and Donchian variants of the loop, or build the vectorized leakage-audit harness first?

📄 🌟 💬 ...

AI Chat • You shouldn't need to pick AI →

Ask anything, create anything

+ Claude Opus 4.8 ▾

🗣️ Speak